Homework 05: Shortest paths, maximum flow, and duality

ISyE 524 — Fall 2026
Due: Monday, October 12, 2026, at 11:59 p.m. (Madison time).

Complete all three problems, which use the network and LP-duality material from Lectures 9–10. For Problems 1–2, use the standard minimum-cost network flow formulation from `mcnf.ipynb`. Write its specialization for each problem before adapting and reusing the Julia/JuMP model. Explain the data, variable domains, objective, constraints, and relevant units. Use continuous variables. Problem 3 requires mathematical reasoning; code is not required.

Submit one PDF to Gradescope and mark which pages belong to each problem. **You will not receive full marks unless you mark the pages.**

Getting started and written answers

Follow Assignments and PDF Submission. In VS Code, use **Tasks: Run Task** to run **ISyE 524: Update course repository**, then **ISyE 524: Start an assignment from its template**, entering `hw05`. Open `student-work/hw05/hw05.ipynb`. An existing working copy will not be overwritten.

For **every written part**, type LaTeX mathematics in Markdown or insert a clear photograph/scan of handwritten work. Include the answer and reasoning together and label the subparts; no duplicate transcription is required. Julia/JuMP implementations belong in code cells.

Use `$x_1 \geq 0$` for inline mathematics and `$$x_1+x_2\leq5$$` for display mathematics. Run the Markdown cell to render it.

For handwritten work, edit a Markdown cell in VS Code, drag in a cropped PNG/JPEG, and choose **Insert Image as Attachment**. Alternatively, put the image beside your working notebook under `student-work/hw05/images/` and insert `![My answer](images/problem1.png)`. Keep external images with the notebook. See Images and handwritten work.

Back up `student-work/hw05/` separately; Git does not back up your answers or images. After completing the notebook, export it with the course PDF task, inspect `submissions/hw05.pdf` for readable mathematics, tables, outputs, and images, and submit that PDF to Gradescope.

Computational setup

Use JuMP and HiGHS for Problems 1–2. Start from the standard minimum-cost network flow notebook (`mcnf.ipynb`), available as `notebooks/12-mcnf.ipynb` in the student repository. Reuse its minimum-cost objective, **outflow − inflow = net supply** balances, and lower/upper arc bounds for every network instance, including maximum flow. Change the network data rather than writing a separate optimization model for each problem. You may wrap the standard model code in a function to reuse it.

Match the notebook's data names: `nodes`, `arcs`, `c`, `b`, `ℓ` (lower bounds), and `u` (upper bounds). The supplied names have suffixes to distinguish instances. Use dictionaries for `b` when node labels are symbols or strings so that `b[i]` is indexed by the node label. Print the termination status and check that the solver found an optimal solution before reading its values. Keep data preparation separate from the model implementation.

The restaurant and maximum-flow data are supplied below. Keep `data/roads.csv` with this notebook for the larger road-network instance in Problem 1(c). All supplied cells run without completed answers. Report the requested summaries rather than printing a large model or every arc flow.

AI tools may help debug your own code, explain syntax or error messages, and critique your work. They should not replace constructing your formulation, implementation, or written solution. Complete the required collaboration and LLM-use statement at the end.

```
using JuMP, HiGHS, CSV, DataFrames, Printf
import MathOptInterface as MOI
```

1: Taste of Madison — and a larger road network

Some friends are visiting Madison. You want to drive from **Short Stack to Alchemy**, possibly stopping at other restaurants along the way, while minimizing total driving time. You do **not** have to visit every restaurant.

The directed connections and travel times are given below. A dash means that there is no direct connection in that direction. Only the listed connections may be used; do not add their reverses.

From / to	G	O	R	I	W	A
S	4	6	5	—	—	—
G	—	1	—	7	—	—
O	—	—	2	5.5	4	—
R	—	—	—	—	5	—
I	—	—	—	—	1	6
W	—	—	—	—	—	8

All times are in minutes. The node labels are **S:** Short Stack; **G:** Great Dane; **O:** Old Fashioned; **R:** Red Sushi; **I:** Ian's at Garver; **W:** Weary Traveler; **A:** Alchemy.

1(a): Formulate and solve the shortest-path problem

Formulate a shortest path from source s to destination t in a directed network with positive arc travel times **as a minimum-cost network flow problem**, using the standard formulation in `mcnf.ipynb`. Let $\mathcal{N}$ and $\mathcal{E}$ denote the nodes and directed arcs, and let the continuous variable x_{ij} denote flow on arc (i, j). Specify the data c_{ij}, b_i, ℓ_{ij}, and u_{ij} for this application and write the resulting indexed LP. Explain the supplies/demand you assign and how a unit of flow represents a route.

Adapt the standard model code from `mcnf.ipynb` to the restaurant data below. Keep this model implementation for reuse in 1(c) and Problem 2. Report an optimal route **in travel order** and its total time. Briefly explain why integer-variable declarations are not needed to obtain an optimal path in this instance.

Symbol	Parameter	Units	Note
c_{ij}	Length of Travel	Minuites	-
b_i	Net supply	Minuites	This must be zero at all nodes but source and sink wich are +1 and -1 respectively
u_{ij}	Maximum arc capacity	Minuites	This must be 1 since we shortest path only sends one supply and demand
ℓ_{ij}	Minimum arc capacity	Minuites	This is zero because we cannot have negitives

$$\min \quad \sum_{(i,j)\epsilon\varepsilon} x_{ij} c_{ij}$$

Subject To:

$$\sum_{(i,j)\epsilon\varepsilon} x_{ij} - \sum_{(j,i)\epsilon\varepsilon} x_{ji} = b_i \quad \forall i \epsilon N$$

$$l_{ij} \leq x_{ij} \leq u_{ij} \quad \forall (i,j) \epsilon \varepsilon$$

Your mathematical formulation and reasoning:

Symbol	Parameter	Units
c_{ij}	Drive Time	Min
b_i	Supply	
u_{ij}	Upper Limit	
l_{ij}	Lower Limit	

$$\min \quad \sum_{(i,j)\epsilon\varepsilon} x_{ij} c_{ij}$$

Subject To:

$$\sum_{(i,j)\epsilon\varepsilon} x_{ij} - \sum_{(i,j)\epsilon\varepsilon} x_{ji} = b_i \quad \forall i \epsilon N$$

$$\ell_{ij} \leq x_{ij} \leq u_{ij} \quad \forall (i,j) \epsilon \varepsilon$$

[Type your answer here, or insert a clear photograph/scan of handwritten work.]

```
nodes_small = [:S, :G, :O, :R, :I, :W, :A]
arcs_small = [(:S,:G), (:S,:O), (:S,:R), (:G,:O), (:G,:I),
           (:O,:R), (:O,:I), (:O,:W), (:R,:W), (:I,:W),
           (:I,:A), (:W,:A)]
c_small = Dict(zip(arcs_small, [4,6,5,1,7,2,5.5,4,5,1,6,8]))
s_small, t_small = :S, :A;
```

```
# Prepare b, ℓ, and u for the restaurant network.
# Adapt the standard mcnf.ipynb model code and keep it for reuse.

ℓ=Dict(zip(arcs_small, [0,0,0,0,0,0,0,0,0,0,0,0]))
u=Dict(zip(arcs_small, [1,1,1,1,1,1,1,1,1,1,1,1]))
b=Dict(zip(nodes_small, [1,0,0,0,0,0,-1]))

MCNF = Model(HiGHS.Optimizer)

@variable(MCNF, ℓ[i,j] <= x[(i,j) in arcs_small] <= u[i,j])

@objective(MCNF, Min,
    sum(c_small[i,j]x[(i,j)] for (i,j) in arcs_small))

@constraint(MCNF, flowbalance[i in nodes_small],
    sum(x[(i,j)] for j in nodes_small if (i,j) in arcs_small) -
    sum(x[(j,i)] for j in nodes_small if (j,i) in arcs_small)
    == b[i])

optimize!(MCNF)
status = termination_status(MCNF)
println("Termination status: ", status)
status == MOI.OPTIMAL || error("HiGHS stopped with status $(status).")
is_solved_and_feasible(MCNF) || error("No feasible optimal solution is available

shortest_time = objective_value(MCNF)

println("Shortest time is: ", shortest_time, " Minuites")
```

```julia
println()

flows = DataFrame(
    flows = value.(x[(i,j)] for (i,j) in arcs_small),
    to_from = collect(arcs_small),
)
```

```
Running HiGHS 1.15.1 (git hash: 04024d701f): Copyright (c) 2026 under MIT licenc
Includes third-party software components, see THIRD_PARTY_NOTICES.md for full de
Using BLAS: blastrampoline
LP has 7 rows; 12 cols; 24 nonzeros
Coefficient ranges:
  Matrix  [1e+00, 1e+00]
  Cost    [1e+00, 8e+00]
  Bound   [1e+00, 1e+00]
  RHS     [1e+00, 1e+00]
Presolving model
6 rows, 11 cols, 22 nonzeros 0s
4 rows, 9 cols, 18 nonzeros 0s
Dependent equations search running on 4 equations with time limit of 1000.00s
Dependent equations search removed 1 rows and 4 nonzeros in 0.00s (limit = 1000.
3 rows, 7 cols, 11 nonzeros 0s
Presolve reductions: rows 3(-4); columns 7(-5); nonzeros 11(-13)
Solving the presolved LP
Using dual simplex solver
  Iteration        Objective     Infeasibilities num(sum)
          0     1.0000000000e+01 Pr: 1(1) 0.0s
          3     1.6500000000e+01 Pr: 0(0) 0.0s

Performed postsolve
Solving the original LP from the solution after postsolve

Model status        : Optimal
Simplex   iterations: 3
Objective value     :  1.6500000000e+01
P-D objective error :  0.0000000000e+00
HiGHS run time      :          0.00
Termination status: OPTIMAL
Shortest time is: 16.5 Minuites
```

	flows	to_from
	Float64	Tuple…
1	1.0	(:S, :G)
2	0.0	(:S, :O)
3	0.0	(:S, :R)
4	1.0	(:G, :O)
5	0.0	(:G, :I)
6	0.0	(:O, :R)
7	1.0	(:O, :I)
8	-0.0	(:O, :W)
9	-0.0	(:R, :W)
10	0.0	(:I, :W)
11	1.0	(:I, :A)
12	0.0	(:W, :A)

Your results and interpretation:

[Type your answer here, or insert a clear photograph/scan of handwritten work.]

In this shortest flow problem I assign 1 supply to the S node and 1 demand to the A node and a 0 net supply at the intemidate nodes inorder to determine the shortest path. This will result in finding the shotest path because the it sets the A matrix in the $ Ax = b$ constraint to be unimodular and thus all nodes will have intiger flows. Since the max flow is 1 and the min flow is 0 this as well as the fact that there is only one unit to flow through the network the result in only finding the shortest path with flows of equal to 1 and all other flows equal to 0. In this problem the optimal path is S-G-O-I-A with a time of 16.5 min.

1(b): Does choosing the cheapest next arc work?

Starting at S, repeatedly take the outgoing arc with the smallest travel time until you reach A. Work this out by hand; you do not need to program this rule. Report its route and total time, compare with 1(a), and explain why a locally cheapest choice can lead to a worse complete route.

Your answer and reasoning:

[Type your answer here, or insert a clear photograph/scan of handwritten work.]

If you take the next cheapest arc in this problem you would travel the path s-g-o-r-w-a with a time of 20 min. This is 4.5 min slower than if the optimal path is selected. This is becasue while each aarc is the shotest it is not the most direct patth, it add an extra stop which results in needing to travel from W to A which is the longest possible arc at 8 minites.

1(c): The same model on a larger road network

The supplied file `data/roads.csv` describes a fixed synthetic road network with **900 intersections and 2,914 directed arcs**. The intersections lie on a 30×30 grid and are numbered row by row. A connected set of roads between neighboring intersections was generated randomly. Each road permits travel in both directions, with positive travel times that may differ by direction.

Each CSV row gives `from`, `to`, and `minutes` for one directed arc. This is already a fixed instance: **do not generate new random data**. Find the shortest route from intersection **1** to intersection **900** by reusing the standard minimum-cost network flow formulation and model implementation from 1(a). Prepare the new node/arc sets, costs, supplies, and lower/upper bounds as separate data.

Report only the **minimum travel time** and the **number of road segments in an optimal route**. Do not print the full model, the entire route, or all arc flows. No new formulation or greedy comparison is required for this part.

```
# Locate this assignment when the kernel starts beside the notebook or
# at the repository root. Prefer an existing personal copy.
assignment_dir = if isfile(joinpath(pwd(), "hw05.ipynb"))
    pwd()
elseif isdir(joinpath(pwd(), "student-work", "hw05"))
    joinpath(pwd(), "student-work", "hw05")
else
    joinpath(pwd(), "assignments", "hw05")
end
road_file = joinpath(assignment_dir, "data", "roads.csv")
isfile(road_file) || error(
    "Missing roads.csv in $(dirname(road_file)). " *
    "Keep the complete data folder beside your HW05 notebook."
)
roads = CSV.read(road_file, DataFrame)
nodes_large = 1:900
arcs_large = [(r.from, r.to) for r in eachrow(roads)]
c_large = Dict((r.from, r.to) => r.minutes for r in eachrow(roads))
s_large, t_large = 1, 900
println("Loaded ", length(nodes_large), " intersections and ",
        length(arcs_large), " directed arcs.")
```

```
Loaded 900 intersections and 2914 directed arcs.
```

```
# Prepare the large-instance MCNF data and reuse the same model.
# Report the travel time and number of road segments.
ℓ_large=Dict(zip(arcs_large, fill(0,length(arcs_large))))
u_large=Dict(zip(arcs_large, fill(1,length(arcs_large))))
b_large=Dict(zip(nodes_large, fill(0,length(nodes_large))))
b_large[nodes_large[1]]=1
b_large[nodes_large[length(nodes_large)]]=-1

MCNF2 = Model(HiGHS.Optimizer)

@variable(MCNF2, ℓ_large[i,j] <= x[(i,j) in arcs_large] <= u_large[i,j])

@objective(MCNF2, Min,
```

```
    sum(c_large[i,j]x[(i,j)] for (i,j) in arcs_large))

@constraint(MCNF2, flowbalance[i in nodes_large],
    sum(x[(i,j)] for j in nodes_large if (i,j) in arcs_large) -
    sum(x[(j,i)] for j in nodes_large if (j,i) in arcs_large)
    == b_large[i])

set_silent(MCNF2)
optimize!(MCNF2)
status = termination_status(MCNF2)
println("Termination status: ", status)
status == MOI.OPTIMAL || error("HiGHS stopped with status $(status).")
is_solved_and_feasible(MCNF2) || error("No feasible optimal solution is availabl

shortest_time_large = objective_value(MCNF2)

println("Shortest time is: ", shortest_time_large, " Minuites")
println()

flows_large = DataFrame(
    flows_large = value.(x[(i,j)] for (i,j) in arcs_large),
    to_from_large = collect(arcs_large),
)

road_segments=0

for (i,j) in arcs_large
    if value.(x[(i,j)]) == 1
        road_segments+=1
    end
end

println("Road segments required: ", road_segments)
```

```
Termination status: OPTIMAL
Shortest time is: 175.39999999999995 Minuites

Road segments required: 58
```

Your reported results:

[Type your answer here, or insert a clear photograph/scan of handwritten work.]

The shortest time for this trip is 175.4 minuites and it will require 58 different road segments.

2: Maximum flow and capacity upgrades

Consider the directed network below. Each arc label is its capacity in units of flow per hour. Find the maximum flow that can be sent from source s to sink t. Intermediate nodes neither create nor consume flow.

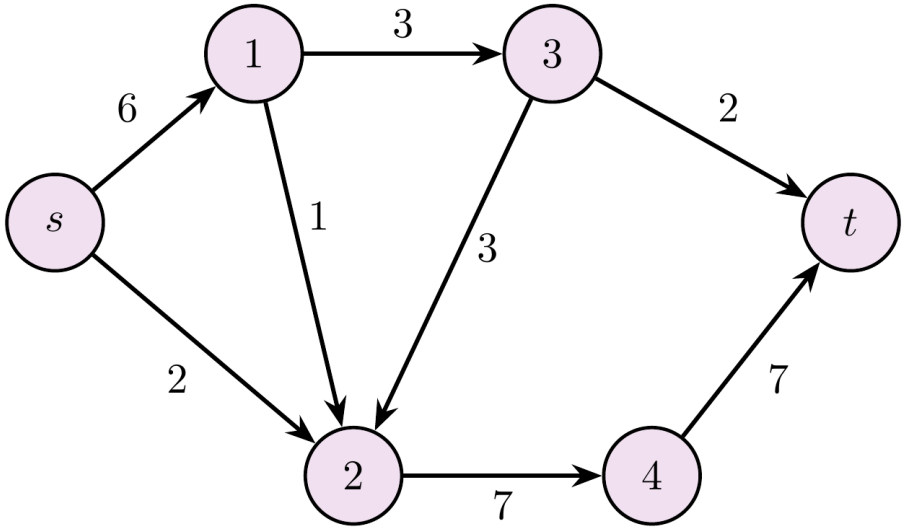


Figure 1: The six-node network, with capacities on its directed arcs.

2(a): Formulate with a feedback arc

Formulate this maximum-flow problem **as a minimum-cost network flow problem** using the same standard formulation from `mcnf.ipynb` and 1(a). Let $\mathcal{N}$ be the nodes and $\mathcal{E}$ the physical arcs, and let the continuous variable x_{ij} denote flow per hour on each arc. Add an artificial **feedback arc** (t, s), giving the augmented arc set $\overline{\mathcal{E}} = \mathcal{E} \cup \{(t, s)\}$.

Specify c_{ij}, b_i, ℓ_{ij}, and u_{ij} on this augmented network and write the resulting indexed **minimization** LP. Choose the costs so that minimizing total cost maximizes throughput. Explain the feedback arc's capacity, why every node has zero net supply, how its flow measures total throughput, and how the minimum objective value relates to the maximum flow.

Your mathematical formulation and reasoning:

[Type your answer here, or insert a clear photograph/scan of handwritten work.]

Symbol	Parameter	Units	Note
c_{ij}	cost	unitless	this is set to be -1 on the sink source arc only and 0 elsewere

Symbol	Parameter	Units	Note
b_i	Net supply	Flow	This must be zero at all nodes becasue we added a feedback arc between sink and source
u_{ij}	Maximum arc capacity	Flow per hr	$u_{ts} = \infty$
ℓ_{ij}	Minimum arc capacity	Flow per hr	This is zero because we cannot have negitives

$$\min \quad \sum_{(i,j)\epsilon\varepsilon} x_{ij} c_{ij}$$

Subject To:

$$\sum_{(i,j)\epsilon\varepsilon} x_{ij} - \sum_{(j,i)\epsilon\varepsilon} x_{ji} = b_i \quad \forall i \epsilon N$$

$$l_{ij} \leq x_{ij} \leq u_{ij} \quad \forall (i,j)\epsilon\varepsilon$$

where the objective function will simplify to:

$$\min \quad x_{ts} * (-1)$$

where the constraints will simplify to:

Subject To:

$$\sum_{(i,j)\epsilon\overline{\mathcal{E}}} x_{ij} - \sum_{(j,i)\epsilon\overline{\mathcal{E}}} x_{ji} = 0 \quad \forall i \epsilon N$$

$$l_{ij} \leq x_{ij} \leq u_{ij} \quad \forall (i,j)\epsilon\overline{\mathcal{E}}$$

The feedback arc can have infinite capacity becasue it is only there to allow us to fully utilize our network. This process works by determineing by minimizing the the negitive of the total amount of product which can leave the source and flow through the network (the value determined by the feedback loop). This forces the optimization to determine the maximum that can flow through the system because if no node can have net supply then it all must flow through.

2(b): Implement and interpret

Reuse the standard minimum-cost network flow model code from 1(a), changing only its input data to the augmented network from 2(a). The supplied arc set contains only the physical arcs; add the feedback arc when preparing the data. Keep the standard minimization objective and flow-balance equations. Report the maximum flow and the flow on each physical arc and the feedback arc. Explain how the flow leaving the source reaches the sink, including any splitting or merging.

```
# Strings "1", ..., "4" label the intermediate nodes in the figure.
nodes_flow = ["s", "1", "2", "3", "4", "t"]
arcs_flow = [("s","1"), ("s","2"), ("1","2"), ("1","3"),
          ("3","2"), ("2","4"), ("3","t"), ("4","t")]
u_flow = Dict(zip(arcs_flow, [6.0,2.0,1.0,3.0,3.0,7.0,2.0,7.0]))
s_flow, t_flow = "s", "t";
```

```
# Prepare the augmented network and its c, b, ℓ, and u data.
# Reuse the MCNF model from 1(a); report throughput and arc flows.

push!(arcs_flow,(t_flow,s_flow))
u_flow2 = merge(u_flow, Dict(("t", "s") => Inf)) #used ai to find this function

c_flow=Dict(zip(arcs_flow, [0,0,0,0,0,0,0,0,-1]))
ℓ_flow=Dict(zip(arcs_flow, [0,0,0,0,0,0,0,0,0]))
b_flow=Dict(zip(nodes_flow, fill(0,length(nodes_flow))))

MCNF3 = Model(HiGHS.Optimizer)

@variable(MCNF3, ℓ_flow[i,j] <= x[(i,j) in arcs_flow] <= u_flow2[i,j])

@objective(MCNF3, Min,
    sum(c_flow[i,j]x[(i,j)] for (i,j) in arcs_flow))

@constraint(MCNF3, flowbalance[i in nodes_flow],
    sum(x[(i,j)] for j in nodes_flow if (i,j) in arcs_flow) -
    sum(x[(j,i)] for j in nodes_flow if (j,i) in arcs_flow)
    == b_flow[i])

optimize!(MCNF3)
status = termination_status(MCNF3)
println("Termination status: ", status)
status == MOI.OPTIMAL || error("HiGHS stopped with status $(status).")
is_solved_and_feasible(MCNF3) || error("No feasible optimal solution is availabl

greatest_flow = -objective_value(MCNF3)

println("Greatest Flow is: ", greatest_flow, " Products per Hr")
println()
```

```
flows_flow = DataFrame(
    flows_flow = value.(x[(i,j)] for (i,j) in arcs_flow),
    to_from_flow = collect(arcs_flow),
)
```

```
Running HiGHS 1.15.1 (git hash: 04024d701f): Copyright (c) 2026 under MIT licenc
Includes third-party software components, see THIRD_PARTY_NOTICES.md for full de
Using BLAS: blastrampoline
LP has 6 rows; 9 cols; 18 nonzeros
Coefficient ranges:
  Matrix  [1e+00, 1e+00]
  Cost    [1e+00, 1e+00]
  Bound   [1e+00, 7e+00]
  RHS     [0e+00, 0e+00]
Presolving model
5 rows, 8 cols, 16 nonzeros 0s
3 rows, 6 cols, 12 nonzeros 0s
2 rows, 5 cols, 6 nonzeros 0s
0 rows, 1 cols, 0 nonzeros 0s
0 rows, 0 cols, 0 nonzeros 0s
Presolve reductions: rows 0(-6); columns 0(-9); nonzeros 0(-18) - Reduced to emp
Performed postsolve
Solving the original LP from the solution after postsolve

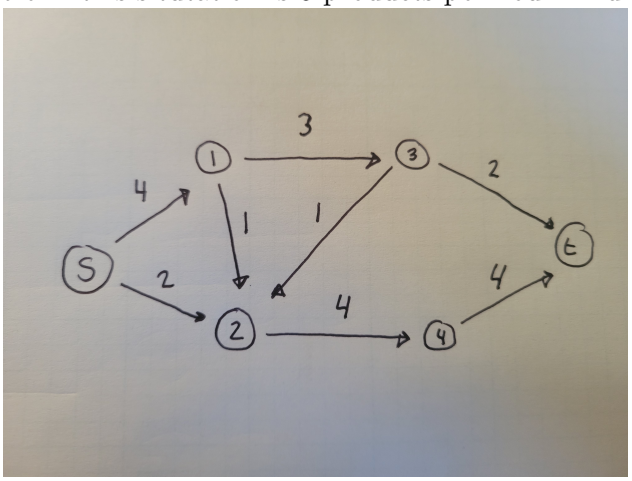
Model status        : Optimal
Objective value     : -6.0000000000e+00
P-D objective error :  0.0000000000e+00
HiGHS run time      :          0.00
Termination status: OPTIMAL
Greatest Flow is: 6.0 Products per Hr
```

	flows_flow	to_from_flow
	Float64	Tuple…
1	4.0	(”s”, ”1”)
2	2.0	(”s”, ”2”)
3	1.0	(”1”, ”2”)
4	3.0	(”1”, ”3”)
5	1.0	(”3”, ”2”)
6	4.0	(”2”, ”4”)
7	2.0	(”3”, ”t”)
8	4.0	(”4”, ”t”)
9	6.0	(”t”, ”s”)

Your results and interpretation:

[Type your answer here, or insert a clear photograph/scan of handwritten work.]

The greatest flow acheivable in this situtation is 6 products per hour. A diagram is show below of the maximum flow path.



2(c): Which capacity upgrade helps?

Consider three separate experiments, each starting from the **original** capacities:

1. Increase the capacity of $(s, 1)$ from 6 to 7.
2. Increase the capacity of $(1, 3)$ from 3 to 4.
3. Increase the capacity of $(4, t)$ from 7 to 8.

For each experiment, change only the relevant upper-bound data and re-solve with the same minimum-cost network flow model. Using your **original optimal solution from 2(b)**, compare the increase in maximum flow with the upgraded arc's **unused capacity**, defined as $u_{ij} - x^0_{ij}$, where x^0_{ij} is its flow in that original solution. Report a table with the arc's original optimal flow, original capacity, unused capacity, new maximum throughput, and improvement over the base case. Use the same original solution for all three comparisons.

What relationship do you observe between unused capacity and the effect of an upgrade? Which upgrade should be chosen if all three have the same cost and the goal is to increase throughput? Explain the results using node balances and capacity limits. You do not need minimum-cut theory or the dual of the maximum-flow LP.

```
# Use the original solution from 2(b) to compute unused capacities.
# Change the relevant upper bound in each independent experiment.
# Reuse the MCNF model and report the requested comparison table.
original_flow1=value.(x[("s","1")] )
original_flow2=value.(x[("1","3")] )
original_flow3=value.(x[("4","t")] )
original_capacity1=u_flow2[("s","1")]
original_capacity2=u_flow2[("1","3")]
original_capacity3=u_flow2[("4","t")]

u_flow2[("s","1")] = 7
```

```
MCNF3 = Model(HiGHS.Optimizer)

@variable(MCNF3, ℓ_flow[i,j] <= x[(i,j) in arcs_flow] <= u_flow2[i,j])

@objective(MCNF3, Min,
    sum(c_flow[i,j]x[(i,j)] for (i,j) in arcs_flow))

@constraint(MCNF3, flowbalance[i in nodes_flow],
    sum(x[(i,j)] for j in nodes_flow if (i,j) in arcs_flow) -
    sum(x[(j,i)] for j in nodes_flow if (j,i) in arcs_flow)
    == b_flow[i])

optimize!(MCNF3)
status = termination_status(MCNF3)
println("Termination status: ", status)
status == MOI.OPTIMAL || error("HiGHS stopped with status $(status).")
is_solved_and_feasible(MCNF3) || error("No feasible optimal solution is availabl

greatest_flow = -objective_value(MCNF3)

println("Greatest Flow is: ", greatest_flow, " Products per Hr")
println()
unused_capacity1= original_capacity1-original_flow1

println("Unused Capacity: ",unused_capacity1)

flows_flow = DataFrame(
    flows_flow = value.(x[(i,j)] for (i,j) in arcs_flow),
    to_from_flow = collect(arcs_flow),
)
```

```
Running HiGHS 1.15.1 (git hash: 04024d701f): Copyright (c) 2026 under MIT licenc
Includes third-party software components, see THIRD_PARTY_NOTICES.md for full de
Using BLAS: blastrampoline
LP has 6 rows; 9 cols; 18 nonzeros
Coefficient ranges:
  Matrix  [1e+00, 1e+00]
  Cost    [1e+00, 1e+00]
  Bound   [1e+00, 7e+00]
  RHS     [0e+00, 0e+00]
Presolving model
5 rows, 8 cols, 16 nonzeros 0s
3 rows, 6 cols, 12 nonzeros 0s
2 rows, 5 cols, 6 nonzeros 0s
0 rows, 1 cols, 0 nonzeros 0s
0 rows, 0 cols, 0 nonzeros 0s
```

```
Presolve reductions: rows 0(-6); columns 0(-9); nonzeros 0(-18) - Reduced to emp
Performed postsolve
Solving the original LP from the solution after postsolve

Model status        : Optimal
Objective value     : -6.0000000000e+00
P-D objective error :  0.0000000000e+00
HiGHS run time      :          0.00
Termination status: OPTIMAL
Greatest Flow is: 6.0 Products per Hr

Unused Capacity: 2.0
```

	flows_flow	to_from_flow
	Float64	Tuple…
1	4.0	(”s”, ”1”)
2	2.0	(”s”, ”2”)
3	1.0	(”1”, ”2”)
4	3.0	(”1”, ”3”)
5	1.0	(”3”, ”2”)
6	4.0	(”2”, ”4”)
7	2.0	(”3”, ”t”)
8	4.0	(”4”, ”t”)
9	6.0	(”t”, ”s”)

```
u_flow2[("s", "1")] = 6
u_flow2[("1", "3")] = 4


MCNF3 = Model(HiGHS.Optimizer)

@variable(MCNF3, ℓ_flow[i,j] <= x[(i,j) in arcs_flow] <= u_flow2[i,j])

@objective(MCNF3, Min,
    sum(c_flow[i,j]x[(i,j)] for (i,j) in arcs_flow))

@constraint(MCNF3, flowbalance[i in nodes_flow],
    sum(x[(i,j)] for j in nodes_flow if (i,j) in arcs_flow) -
    sum(x[(j,i)] for j in nodes_flow if (j,i) in arcs_flow)
    == b_flow[i])

optimize!(MCNF3)
status = termination_status(MCNF3)
println("Termination status: ", status)
status == MOI.OPTIMAL || error("HiGHS stopped with status $(status).")
is_solved_and_feasible(MCNF3) || error("No feasible optimal solution is availabl
```

```julia
greatest_flow = -objective_value(MCNF3)

println("Greatest Flow is: ", greatest_flow, " Products per Hr")
println()
unused_capacity2=original_capacity2-original_flow2

println("Unused Capacity: ",unused_capacity2)
flows_flow = DataFrame(
    flows_flow = value.(x[(i,j)] for (i,j) in arcs_flow),
    to_from_flow = collect(arcs_flow),
)
```

```
Running HiGHS 1.15.1 (git hash: 04024d701f): Copyright (c) 2026 under MIT licenc
Includes third-party software components, see THIRD_PARTY_NOTICES.md for full de
Using BLAS: blastrampoline
LP has 6 rows; 9 cols; 18 nonzeros
Coefficient ranges:
  Matrix  [1e+00, 1e+00]
  Cost    [1e+00, 1e+00]
  Bound   [1e+00, 7e+00]
  RHS     [0e+00, 0e+00]
Presolving model
5 rows, 8 cols, 16 nonzeros 0s
3 rows, 6 cols, 12 nonzeros 0s
2 rows, 5 cols, 6 nonzeros 0s
0 rows, 1 cols, 0 nonzeros 0s
0 rows, 0 cols, 0 nonzeros 0s
Presolve reductions: rows 0(-6); columns 0(-9); nonzeros 0(-18) - Reduced to emp
Performed postsolve
Solving the original LP from the solution after postsolve

Model status        : Optimal
Objective value     : -7.0000000000e+00
P-D objective error :  0.0000000000e+00
HiGHS run time      :          0.00
Termination status: OPTIMAL
Greatest Flow is: 7.0 Products per Hr

Unused Capacity: 0.0
```

	flows_flow	to_from_flow
	Float64	Tuple…
1	5.0	(”s”, ”1”)
2	2.0	(”s”, ”2”)
3	1.0	(”1”, ”2”)
4	4.0	(”1”, ”3”)
5	2.0	(”3”, ”2”)
6	5.0	(”2”, ”4”)
7	2.0	(”3”, ”t”)
8	5.0	(”4”, ”t”)
9	7.0	(”t”, ”s”)

```
original_flow=value.(x[("4","t")] )

u_flow2[("1", "3")] = 3
u_flow2[("4", "t")] = 8


MCNF3 = Model(HiGHS.Optimizer)

@variable(MCNF3, ℓ_flow[i,j] <= x[(i,j) in arcs_flow] <= u_flow2[i,j])

@objective(MCNF3, Min,
    sum(c_flow[i,j]x[(i,j)] for (i,j) in arcs_flow))

@constraint(MCNF3, flowbalance[i in nodes_flow],
    sum(x[(i,j)] for j in nodes_flow if (i,j) in arcs_flow) -
    sum(x[(j,i)] for j in nodes_flow if (j,i) in arcs_flow)
    == b_flow[i])

optimize!(MCNF3)
status = termination_status(MCNF3)
println("Termination status: ", status)
status == MOI.OPTIMAL || error("HiGHS stopped with status $(status).")
is_solved_and_feasible(MCNF3) || error("No feasible optimal solution is availabl

greatest_flow = -objective_value(MCNF3)

println("Greatest Flow is: ", greatest_flow, " Products per Hr")
println()
unused_capacity3=original_capacity3-original_flow3

println("Unused Capacity: ",unused_capacity3)
flows_flow = DataFrame(
    flows_flow = value.(x[(i,j)] for (i,j) in arcs_flow),
    to_from_flow = collect(arcs_flow),
)
```

```
Running HiGHS 1.15.1 (git hash: 04024d701f): Copyright (c) 2026 under MIT licenc
Includes third-party software components, see THIRD_PARTY_NOTICES.md for full de
Using BLAS: blastrampoline
LP has 6 rows; 9 cols; 18 nonzeros
Coefficient ranges:
  Matrix  [1e+00, 1e+00]
  Cost    [1e+00, 1e+00]
  Bound   [1e+00, 8e+00]
  RHS     [0e+00, 0e+00]
Presolving model
5 rows, 8 cols, 16 nonzeros 0s
3 rows, 6 cols, 12 nonzeros 0s
2 rows, 5 cols, 6 nonzeros 0s
0 rows, 1 cols, 0 nonzeros 0s
0 rows, 0 cols, 0 nonzeros 0s
Presolve reductions: rows 0(-6); columns 0(-9); nonzeros 0(-18) - Reduced to emp
Performed postsolve
Solving the original LP from the solution after postsolve

Model status        : Optimal
Objective value     : -6.0000000000e+00
P-D objective error :  0.0000000000e+00
HiGHS run time      :          0.00
Termination status: OPTIMAL
Greatest Flow is: 6.0 Products per Hr

Unused Capacity: 3.0
```

	flows_flow	to_from_flow
	Float64	Tuple…
1	4.0	(”s”, ”1”)
2	2.0	(”s”, ”2”)
3	1.0	(”1”, ”2”)
4	3.0	(”1”, ”3”)
5	1.0	(”3”, ”2”)
6	4.0	(”2”, ”4”)
7	2.0	(”3”, ”t”)
8	4.0	(”4”, ”t”)
9	6.0	(”t”, ”s”)

Your answer and reasoning:

[Type your answer here, or insert a clear photograph/scan of handwritten work.]

What relationship do you observe between unused capacity and the effect of an upgrade? Which upgrade should be chosen if all three have the same cost and the goal is to increase throughput? Explain the results using node balances and capacity limits. You do not need minimum-cut theory or the dual of the maximum-flow LP.

Arc	Original Optimal Flow	Original Capacity	Unused Capacity	New Max throughput	Improvement over base
s->1	4	6	2	4	0
1->3	3	3	0	4	1
4->t	4	7	3	4	0

In this problem it is is only advantagous to increace capacity of an arc if it is already at capacity. This makes sence becasue unless there is a bottleneck eleswere it is most likely that the arcs that are at capacity are the limiting factor. Given the option to upgrade I would choose the increace on arc 1->3 because in increaces the total throughput per hour from 6 products to 7 be reducing the bottleneck that occurs at the 1->3 arc. After the 1->3 arc there is still plenty of capacity on the 3->2 2->4 and 4->t to handle the upgrade. I woud not select the s->1 or 4->t upgrade because both have unused capacity in the original optimal soloution so adding capacity just increaces unused capacity.

3: Duality — bounds and optimality certificates

Consider the primal LP

$$\begin{aligned} \max \quad & 3x_1 + x_2 \\ \text{s.t.} \quad & 4x_1 + x_2 \le 7, \\ & 5x_1 + 2x_2 \le 10, \\ & x_1, x_2 \ge 0. \end{aligned}$$

This problem is about mathematical reasoning. **No code is required.** In 3(c), use complementary slackness to construct a dual solution complementary to the given primal solution. In each optimality argument, check feasibility and explain how equality of objective values and weak duality certify optimality. A solver result is not a substitute for the requested reasoning.

3(a): Form the dual

Write the dual LP, using y_1, y_2 for the dual variables. Include the objective, every constraint, and the sign restrictions.

Your answer and reasoning:

[Type your answer here, or insert a clear photograph/scan of handwritten work.]

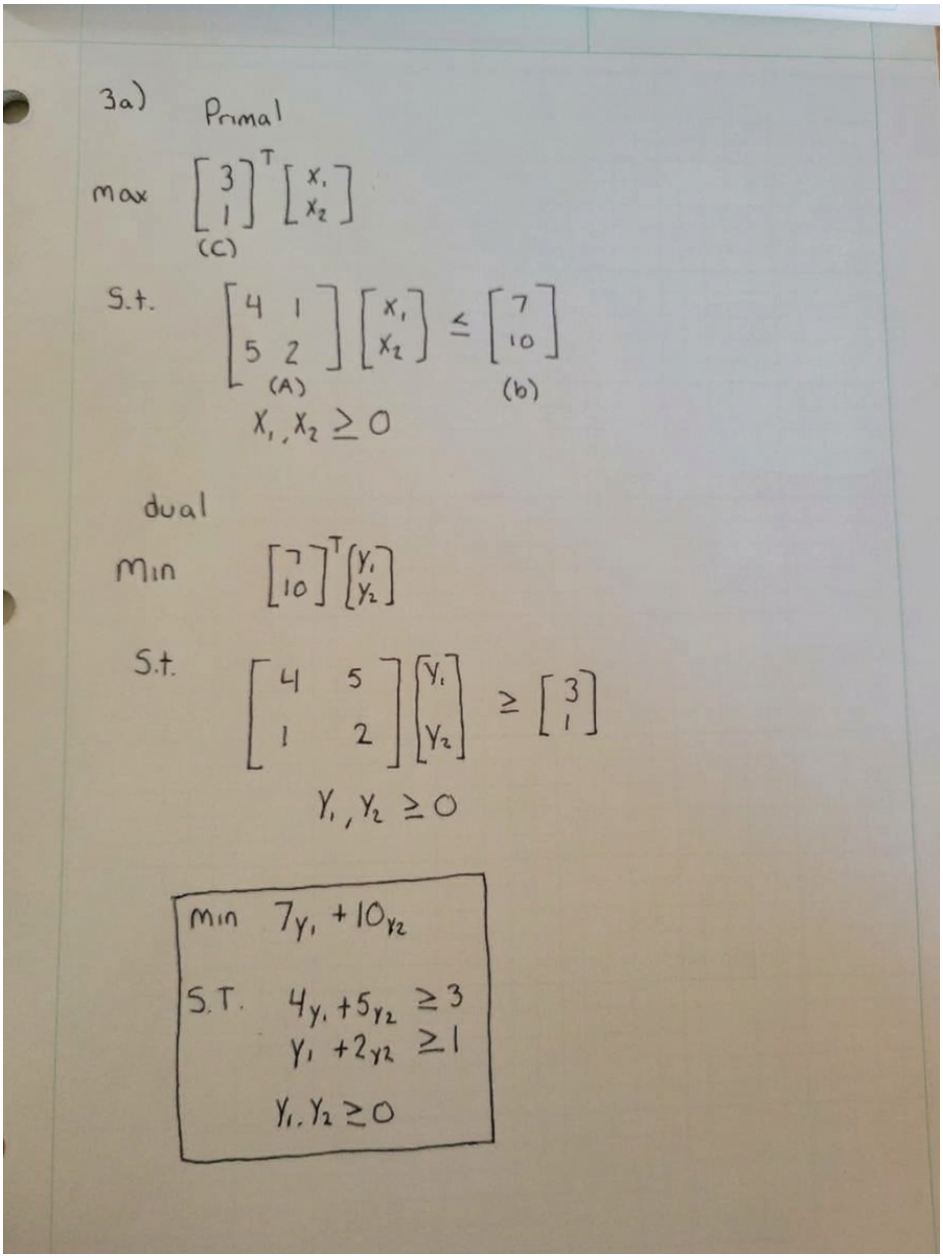


Figure 2: Problem 3a.jpeg

3(b): What weak duality actually says

For **any** feasible primal vector x and feasible dual vector y, prove directly that

$$3x_1 + x_2 \leq 7y_1 + 10y_2.$$

Build the argument from the primal and dual constraints, and identify where the nonnegativity of x and of y is needed.

Now check the candidate vectors $\bar{x} = (1, 1)$ and $\bar{y} = (1/2, 1/4)$. Use their objective values to give an interval containing the optimal primal value. Do these bounds permit a feasible primal vector with

objective value 6.5? Do they prove that objective value 6 is attainable? Explain both answers using only the information from these two feasible vectors.

Your answer and reasoning:

[Type your answer here, or insert a clear photograph/scan of handwritten work.]

Proof Below

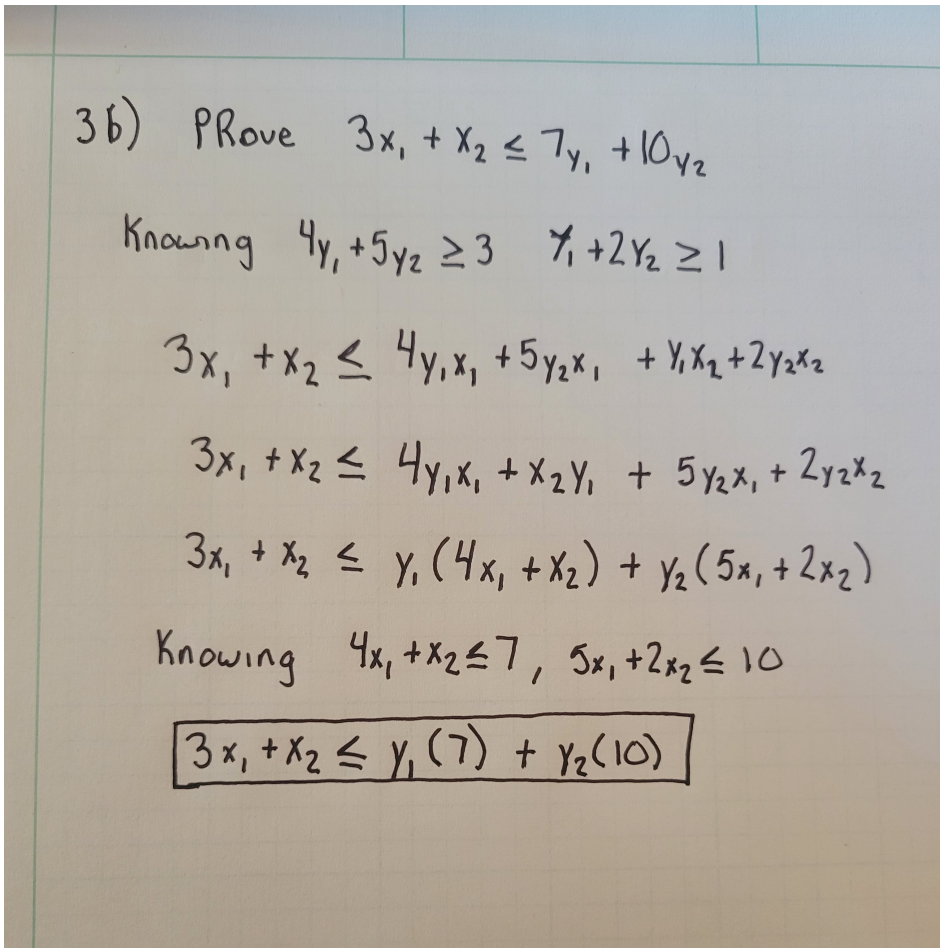


Figure 3: 1000006289.jpg

X and Y must be non negitive for the constraints to be multiplied in and the inequaltiy to hold true. Otherwise the inequality would flip causing this statment to not hold true. This occurs dirreclty after the first knowing stament in my written work above.

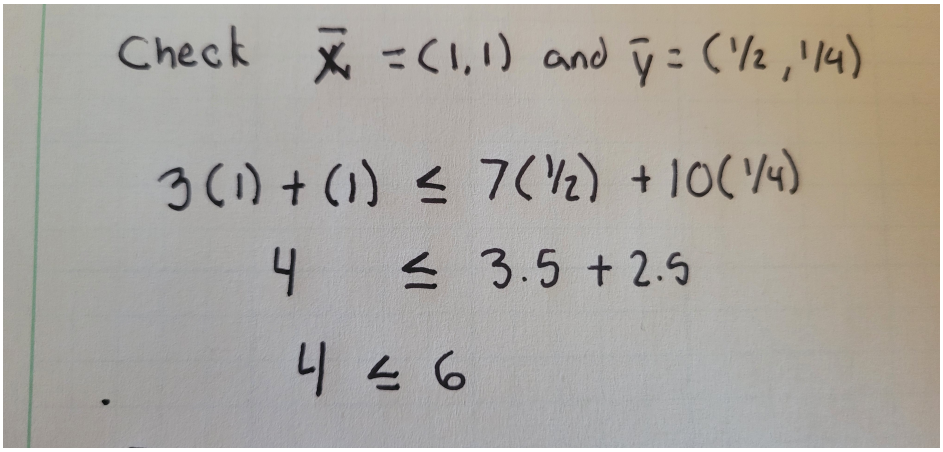


Figure 4: 1000006290.jpg

These bounds allow a objective value of 6 to exist but not 6.5 becasue the two vectors $\bar{x} = (1, 1)$ and $\bar{y} = (1/2, 1/4)$ both exist within their feasable spaces thus they bound the soloution. The optimal soloution must fall within these bounds 6.5 is greater than the upper bound of 6.

3(c): Find a complementary dual solution and certify optimality

Verify that the proposed primal vector $\hat{x} = (4/3, 5/3)$ is feasible. Use the complementary slackness conditions to find a dual-feasible solution $\hat{y}$ complementary to $\hat{x}$. Explain which conditions determine $\hat{y}$, verify feasibility and all complementary slackness conditions, and use equality of objective values and weak duality to prove that both solutions are optimal. Explain why satisfying the complementary slackness equations without checking feasibility would be insufficient.

Your answer and reasoning:

[Type your answer here, or insert a clear photograph/scan of handwritten work.]

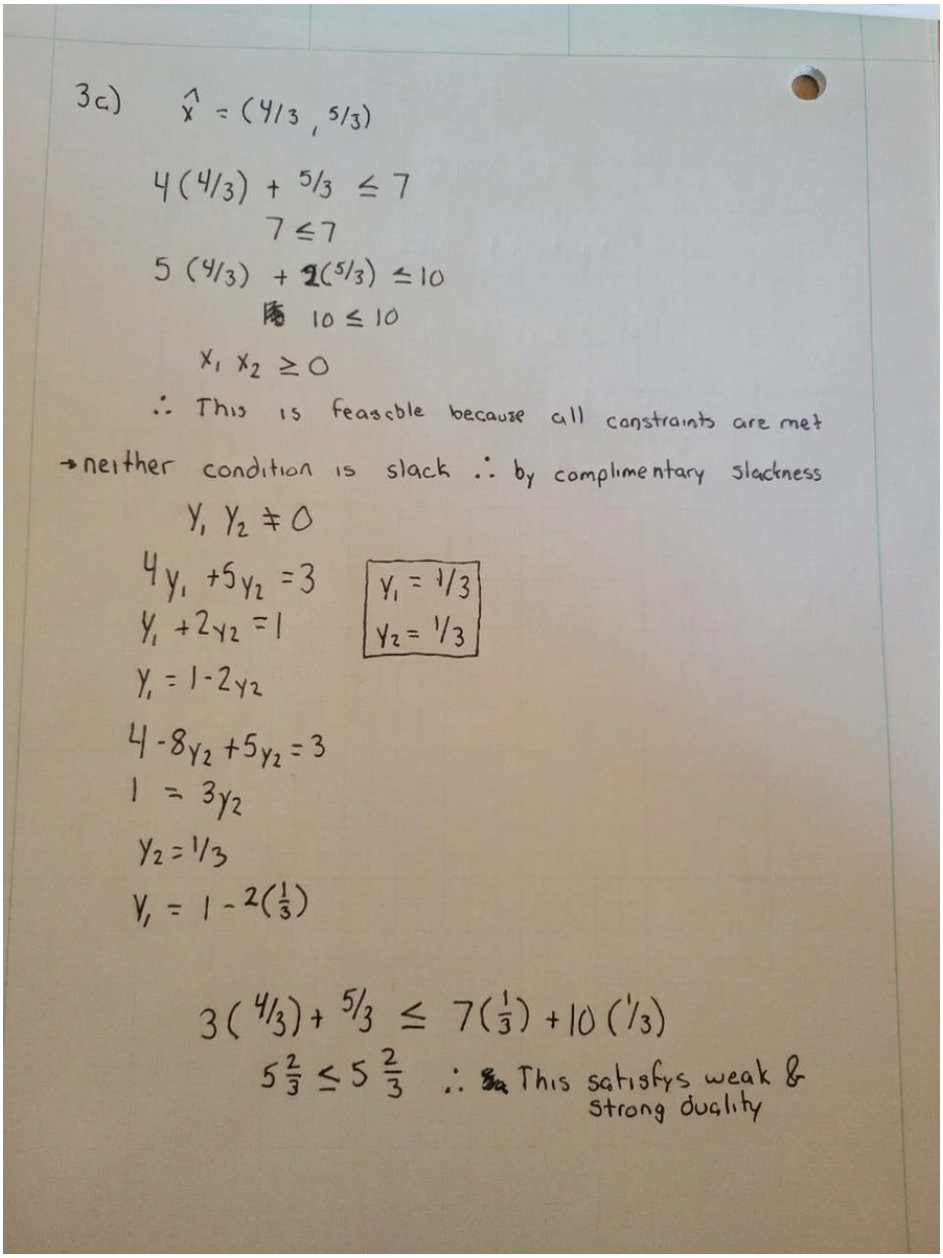


Figure 5: Problem 3c.jpg

You cannot just check complementary slackness in this case without checking feasablity beceause a dual only shares feasable soloutions with its primal. Thus if the value lands outside the feasable region it may still exist in the dual but it isnt a soloution becasue it is infeasable in the primal.

3(d): More of the second resource

Change the right-hand side of the **second primal constraint from 10 to 12**. All other data stay fixed.

Determine whether the dual vector from 3(c) is still feasible. Use it and weak duality to obtain an upper bound on the new optimal primal value. Find a feasible primal point that attains this bound and prove optimality by checking its constraints and objective.

Your answer and reasoning:

[Type your answer here, or insert a clear photograph/scan of handwritten work.]

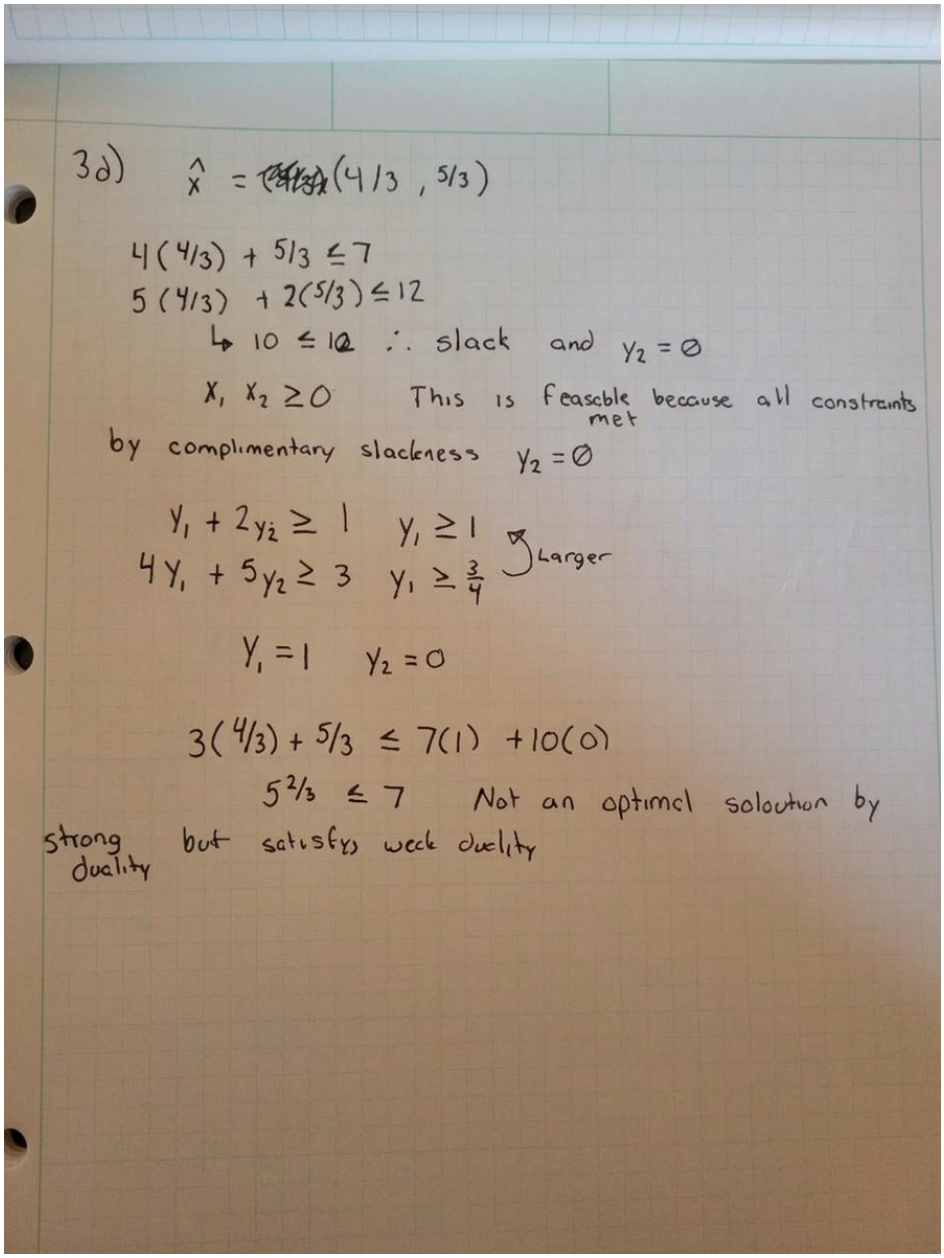


Figure 6: Problem 3d.jpg

Collaboration and LLM-use statement — required

Complete this statement as the **very last part of your homework**. Be specific enough that a reader can understand what work you did yourself and how others or LLMs helped you learn.

- **People:** Give the name of every person you collaborated with, identify the problem/subpart you discussed, and explain the nature of the collaboration. If you worked alone, explicitly state that you did not collaborate with anyone.
- **LLMs/tools:** Name every LLM or AI assistant you used and identify the model/version as precisely as available. If the interface did not show a model/version, say so. If you used none, explicitly state that you did not use an LLM or AI assistant.
- **Specific assistance and learning:** For each tool, identify the problem/subpart, describe what you asked it to help with, what suggestions you used or rejected, and how the interaction helped you understand the mathematics or implementation. Explain how you checked the suggestions. "I used ChatGPT for help" is not sufficiently specific.

Disclosure does not replace the requirement that your submitted formulation, implementation, and explanation be your own work and that you understand them.

People and collaboration:

[Names, problem/subparts, and what you discussed; or explicitly state that you worked alone.]

I worked alone

LLMs/tools and models:

[Tool names and models/versions, or explicitly state that you used none.]

I used gemini 3.5 flash lite

How I used the assistance, what I learned, and how I checked it:

[Specific details for each tool and relevant problem/subpart; state not applicable if you used none.]

I used a llm to find the merge function for dictionarys used in problem 2. I had it direct me to the julia documentation and read up on the usage of it to verify the correctness of its responce.